BrainAgentictho n 2026 | Medical AI Workstream

الدليل العملي الأول: فحص بيانات MRI من الصفر

Task01_BrainTumour | Dataset Audit + Validation | Windows / VS Code

عبدالرحمن + رؤى | إصدار 15 أكتوبر 2026

الهدف: إعادة فحص Task01_BrainTumour Dataset من البداية، بطريقة قابلة للتكرار، دون تغيير ملفات الأصل، ودون اتصال بالإنترنت أثناء الفحص.

من أين نبدأ؟ بحسب لقطة التشغيل السابقة نجح فحص حالة واحدة: MRI shape = (240,240,155,4)، القناع (240,240,155)، Labels = 0,1,2,3، ولا توجد NaN/Inf في تلك الحالة. هذا **لا يعني** سلامة كل 484 حالة.

1. ما الذي نعرفه عن الداتا؟

العنصر	القيمة المعروفة من dataset.json
الاسم	BRATS - Medical Segmentation Decathlon Task01
التدريب	484 حالة مذكورة في manifest
الاختبار	266 حالة مذكورة في manifest، بدون أقنعة عامة في imagesTs
قنوات الصورة	0: FLAIR | 1: T1w | 2: T1gd | 3: T2w
الملصقات	0: background | 1: edema | 2: non-enhancing tumor | 3: enhancing tumor
تنسيق الملفات	nii.gz (NIfTI)، MRI. متعددة القنوات
الحدود	Glioma tumor segmentation وليست بيانات Normal أو عشر فئات تشخيص

2. الملفات التي يجب أن تكون موجودة

```
Task01_BrainTumour/
  dataset.json
  imagesTr/
  labelsTr/
  imagesTs/
  inspect_dataset.py     # optional: previous one-case test
  validate_all_cases.py  # new: full training audit
```

> ملفات `._imagesTr` و `._dataset.json` إذا ظهرت، فهي غالبًا ملفات أرشفة ثانوية؛ لا تدخل في التدريب. احتفظ بالمجلدات الأصلية ولا تعيد تسمية الصور.

3. خطوات الإعداد على Windows / VS Code

أ. افتح المجلد

افتح `Task01_BrainTumour` في VS Code ثم افتح PowerShell من Terminal. تأكد أن المسار الحالي هو مجلد الداتا نفسه.

ب. افحص Python

```
py --version
python --version
```

لديك Python 3.14 ظهر في لقطة الشاشة السابقة. لفحص NIfTI فقط يمكن استخدام البيئة التي نجحت، لكن لتدريب MONAI/PyTorch لاحقًا يفضّل إعداد Python 3.11 أو 3.12 بعد التحقق من توافق الإصدارات.

ج. أنشئ بيئة مستقلة، إن لم تكن قد أنشأتها

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install numpy nibabel
```

> إذا كانت حزمة pip موجودة في Python الذي استخدمته سابقًا، تستطيع مؤقتًا الاستمرار به. الفحص يحتاج `nibabel` و `numpy` فقط؛ MONAI ليس شرطًا في هذه الخطوة.

د. تحقق أن جميع الملفات الأساسية موجودة

```
Get-ChildItem .\dataset.json
(Get-ChildItem .\imagesTr\*.nii.gz).Count
(Get-ChildItem .\labelsTr\*.nii.gz).Count
(Get-ChildItem .\imagesTs\*.nii.gz).Count
```

الأعداد الناتجة يجب مقارنتها بما في `dataset.json` . لا تعتبر هذا وحده إثبات سلامة الملفات.

4. شغّل الفحص الكامل

ضع الملف المرفق `validate_all_cases.py` داخل مجلد الداتا. الأمر التالي يفحص جميع مدخلات التدريب دون الكتابة فوق الصور:

```
.\.venv\Scripts\python.exe .\validate_all_cases.py --data . --out validation_report.json
```

إذا لم تستخدم البيئة الافتراضية، يمكنك التشغيل باستخدام مسار Python المثبت الذي عمل في التجربة السابقة.

```
& "C:\Users\User\AppData\Local\Python\pythoncore-3.14-64\python.exe" .\validate_all_cases.py --
data . --out validation_report.json
```

5. ماذا يفحص السكربت؟

فحص	لماذا؟
وجود ملفات الصور والأقنعة	تجنب فقد حالات أو توقف التدريب لاحقًا
عدد القنوات = 4	مطابقة صيغة MRI متعددة الـmodalities
الأبعاد المكانية	أن تكون XYZ للصورة مساوية لأبعاد القناع
Affine + Spacing	التأكد من مواءمة الصورة والقناع مكانيًا
قيم القناع	حصرها في {0,1,2,3}
NaN / Inf	اكتشاف القيم غير الصالحة قبل التطبيع
IDs المكررة	تنبيه إلى تكرار أسماء الحالات بين قوائم التدريب والاختبار
أخطاء القراءة	كشف الملفات غير القابلة للفتح أو الفاسدة

حدود مهمة: السكربت لا يثبت عدم وجود تسرب مرضى من خلال معرفات مخفية أو مشتقة، ولا يفحص جميع صور الاختبار عدديًا، ولا يحكم على جودة أقنعة الأورام طبيًا. ذلك يحتاج فحوصًا إضافية ومراجعة بصرية.

6. كيف نقرأ النتيجة؟

```
[1/484] BRATS_457: PASS
...
SUMMARY: {
  "actual_training_entries": 484,
  "passed": 484,
  "failed": 0,
  "training_duplicates": [],
  "train_test_case_id_overlap": []
}
```

هذا **مثال توضيحي لشكل الإخراج وليس نتيجة فعلية**. لا تتوقع أن تساوي أرقامك المثال إلا إذا تحقق الفحص منها. التقرير المفصل سيُحفظ في `validation_report.json`.

7. ماذا لو ظهرت أخطاء؟

الخطأ	الإجراء
missing image / label	تأكد من فك الضغط كاملًا ومن المسار؛ لا تصنع ملفًا بديلًا
MRI shape not 4D	اعزل الحالة وسجلها للمراجعة؛ لا تغير ترتيب المحاور عشوائيًا
affine mismatch	قارن الـorientation والـheader بصريًا قبل أي resampling
labels outside 0,1,2,3	أوقف استخدام الحالة حتى تتأكد من النسخة ومخطط الملصقات
NaN / Inf	اعزل الحالة؛ لا تعوضها بصفر دون توثيق
Python / ModuleNotFoundError	شغّل من نفس البيئة المثبت فيها numpy وnibabel

8. دور كل واحد اليوم

عبدالرحمن	رؤى
تشغيل التحقق على الحالات الـ484 وحفظ التقرير	اختيار 5 حالات تدريب مختلفة وفتحها في 3D Slicer
تسجيل إجمالي PASS/FAIL وأسماء الملفات الفاشلة	تأكد بصريًا من تراكب صورة FLAIR والقناع ومناطق Label 1/2/3
عدم تحريك الملفات أو حذف حالات قبل المراجعة	تسجيل أية Masks تبدو غير متوافقة أو مشوهة

9. Checklist قبل البدء بالتدريب

- [ ] التقرير نتج بنجاح، وأي فشل تم شرحه وتوثيقه.
- [ ] تم التحقق من مطابقة MRI وMask مكانيًا.
- [ ] فُهم ترتيب القنوات 0-3 وترميز Labels بدقة.

- [ ] اتُّفق على تقسيم Train/Validation على مستوى الحالة دون تسرب.
- [ ] تمت مراجعة أمثلة بصرية داخل 3D Slicer.
- [ ] بقيت النسخة الأصلية Read-only ولم تتغير.

10. الخطوة التالية بعد نجاح الفحص

ننشىء split ثابتًا موثقًا، ثم نحدد Region Labels (WT/TC/ET) وتعريفاتها بما يتوافق مع مخطط هذه الداتا قبل تدريب nnU-Net، ونبني Baseline Segmentation حقيقي. لا نبدأ تصنيف Normal vs Abnormal من هذه الداتا وحدها لأنها لا تحتوي فئة Normal موثقة.

المراجع والأساس

- ملف `dataset.json` المحلي، كما ظهر في شاشة VS Code المرسلة.
- وثيقة المسابقة الرسمية BrainAgenticthon ACIT26_VF (2).pdf: تحدد Spatial output وDice/HD95 وOffline/Privacy.
- Medical Segmentation Decathlon: `https://medicaldecathlon.com/`
- Nibabel documentation: `https://nipy.org/nibabel/`

ملحق: الكود الكامل لفحص البيانات

هذا هو السكربت المرفق كملف Python مستقل. يمكنك نسخه أو استخدام الملف مباشرة دون إعادة كتابة الكود.

```
#!/usr/bin/env python3
"""Read-only dataset audit for MSD Task01_BrainTumour; no network calls."""
from pathlib import Path
from collections import Counter
import argparse
import json
import traceback
import numpy as np
import nibabel as nib


def inspect_case(root, name, image_rel, label_rel=None):
    result = {"case": name, "passed": False, "errors": [], "warnings": []}
    img_path = (root / image_rel).resolve()
    if not img_path.is_relative_to(root.resolve()):
        result["errors"].append("image path escapes dataset directory")
        return result
    if not img_path.is_file():
        result["errors"].append("missing image: " + image_rel)
        return result
    try:
        img = nib.load(str(img_path))
        sh = img.shape
        result["image_shape"] = list(sh)
        result["spacing"] = [float(x) for x in img.header.get_zooms()[:3]]
        if len(sh) != 4 or sh[-1] != 4:
            result["errors"].append("expected [X,Y,Z,4] MRI")
        if any((not np.isfinite(x) or x <= 0) for x in result["spacing"]):
            result["errors"].append("invalid voxel spacing")
        if not np.all(np.isfinite(img.affine)):
            result["errors"].append("non-finite MRI affine")
        # Iterate modalities individually, avoiding full 4D array materialization.
        if len(sh) == 4:
            for c in range(sh[-1]):
                a = np.asarray(img.dataobj[..., c])
                if not np.isfinite(a).all():
```

```
                    result["errors"].append(f"non-finite intensity in channel {c}")
        if label_rel is not None:
            mask_path = (root / label_rel).resolve()
            if not mask_path.is_relative_to(root.resolve()):
                result["errors"].append("label path escapes dataset directory")
            elif not mask_path.is_file():
                result["errors"].append("missing label: " + label_rel)
            else:
                mask = nib.load(str(mask_path))
                result["label_shape"] = list(mask.shape)
                if tuple(sh[:3]) != tuple(mask.shape):
                    result["errors"].append("image-mask spatial shape mismatch")
                if not np.allclose(img.affine, mask.affine, atol=1e-3, rtol=1e-5):
                    result["errors"].append("image-mask affine mismatch")
                m = np.asarray(mask.dataobj)
                if not np.isfinite(m).all():
                    result["errors"].append("non-finite mask")
                else:
                    labels, counts = np.unique(m, return_counts=True)
                    result["labels_present"] = [int(v) for v in labels]
                    result["label_counts"] = {str(int(k)): int(v) for k, v in zip(labels, counts)}
                    if not np.isin(labels, [0, 1, 2, 3]).all():
                        result["errors"].append("labels outside {0,1,2,3}")
                    if not np.all(labels == np.floor(labels)):
                        result["errors"].append("non-integer mask values")
        result["passed"] = len(result["errors"]) == 0
    except Exception as e:
        result["errors"].append(f"{type(e).__name__}: {e}")
    return result


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", type=Path, default=Path(__file__).resolve().parent)
    ap.add_argument("--out", type=Path, default=Path("validation_report.json"))
    args = ap.parse_args()
```

```
    root = args.data.expanduser().resolve()
    with (root / "dataset.json").open(encoding="utf-8") as f:
        manifest = json.load(f)
    training = manifest.get("training", [])
    testing = manifest.get("test", [])
    results = []
    ids = []
    for i, case in enumerate(training, 1):
        rel = case["image"]
        case_id = Path(rel).name.replace(".nii.gz", "").replace(".nii", "")
        ids.append(case_id)
        res = inspect_case(root, case_id, rel, case.get("label"))
        results.append(res)
        print(f"[{i}/{len(training)}] {case_id}: {'PASS' if res['passed'] else 'FAIL'}",
flush=True)
    test_ids = [Path(x).name.replace(".nii.gz", "").replace(".nii", "") for x in testing]
    overlaps = sorted(set(ids) & set(test_ids))
    duplicates = sorted(k for k, n in Counter(ids).items() if n > 1)
    summary = {
        "expected_training": manifest.get("numTraining"),
        "actual_training_entries": len(training),
        "expected_test": manifest.get("numTest"),
        "actual_test_entries": len(testing),
        "passed": sum(x["passed"] for x in results),
        "failed": sum(not x["passed"] for x in results),
        "training_duplicates": duplicates,
        "train_test_case_id_overlap": overlaps,
        "note": "Test files are not fully validated; public test has no masks. Exact schema is
organizer-dependent."
    }
    report = {"dataset": manifest.get("name"), "summary": summary, "cases": results}
    args.out.parent.mkdir(parents=True, exist_ok=True)
    args.out.write_text(json.dumps(report, indent=2, ensure_ascii=False), encoding="utf-8")
    print("\nSUMMARY:", json.dumps(summary, indent=2))
    print("Saved:", args.out.resolve())
    if summary["failed"] or overlaps or duplicates or len(training) !=
manifest.get("numTraining"):
        raise SystemExit(1)
```

```
if __name__ == "__main__":
    main()
```